

Payment System Design

Complete Guide with Architecture Diagrams

Source: *The Pragmatic Engineer* (Gergely Orosz) · *System Design Interview Vol. 2* (Alex Xu & Sahn Lam)

1M tx / day

~10 TPS

ACID SQL DB

No card storage

Double-entry

Reconciliation

1. Step 1 — Understand the Problem & Requirements

We are designing the **payment backend for an e-commerce platform** (like Amazon). The system handles money movement for every order — receiving money from buyers on behalf of sellers (pay-in) and later sending money to sellers after delivery (pay-out). Credit cards are processed via third-party PSPs (Stripe, Braintree, Square). We never store raw card numbers — that responsibility is delegated to the PSP.

Functional Requirements

Requirement	Detail
Pay-in flow	Payment system receives money from customers on behalf of sellers
Pay-out flow	Payment system sends money to sellers around the world after delivery
Credit card payments	Processed via third-party PSP (Stripe, Braintree, Square)
No card storage	We never store card numbers — PSP-hosted page handles all sensitive data
Currency	Single currency in this design; real systems handle multi-currency

Non-Functional Requirements

Requirement	Detail
Reliability & fault tolerance	Failed payments must be carefully handled; no money lost
Reconciliation	Async process verifying consistency across internal services and PSP
Exactly-once delivery	A payment must never be processed more than once
Consistency	Internal state (ledger/wallet/DB) must remain consistent after any failure

Back-of-the-Envelope Estimation

$1,000,000 \text{ transactions / day} \div 86,400 \text{ seconds} \approx \mathbf{10 \text{ TPS}}$. This is not a throughput problem — a typical relational database handles thousands of TPS. The real challenge is **correctness**: preventing double-charges, handling partial failures, and keeping all distributed state consistent.

Interview tip: 10 TPS is trivially low. Signal to your interviewer that you understand the challenge is correctness and fault-tolerance, not raw throughput. Candidates who reach for Kafka for 10 TPS miss the point.

2. Step 2 — High-Level Architecture

The payment flow is split into two macro-flows. In the **pay-in flow**, money moves from the buyer's credit card into the platform's bank account (Amazon acts as custodian). In the **pay-out flow**, money moves from the platform's bank account to each seller's bank account once the delivery condition is satisfied.

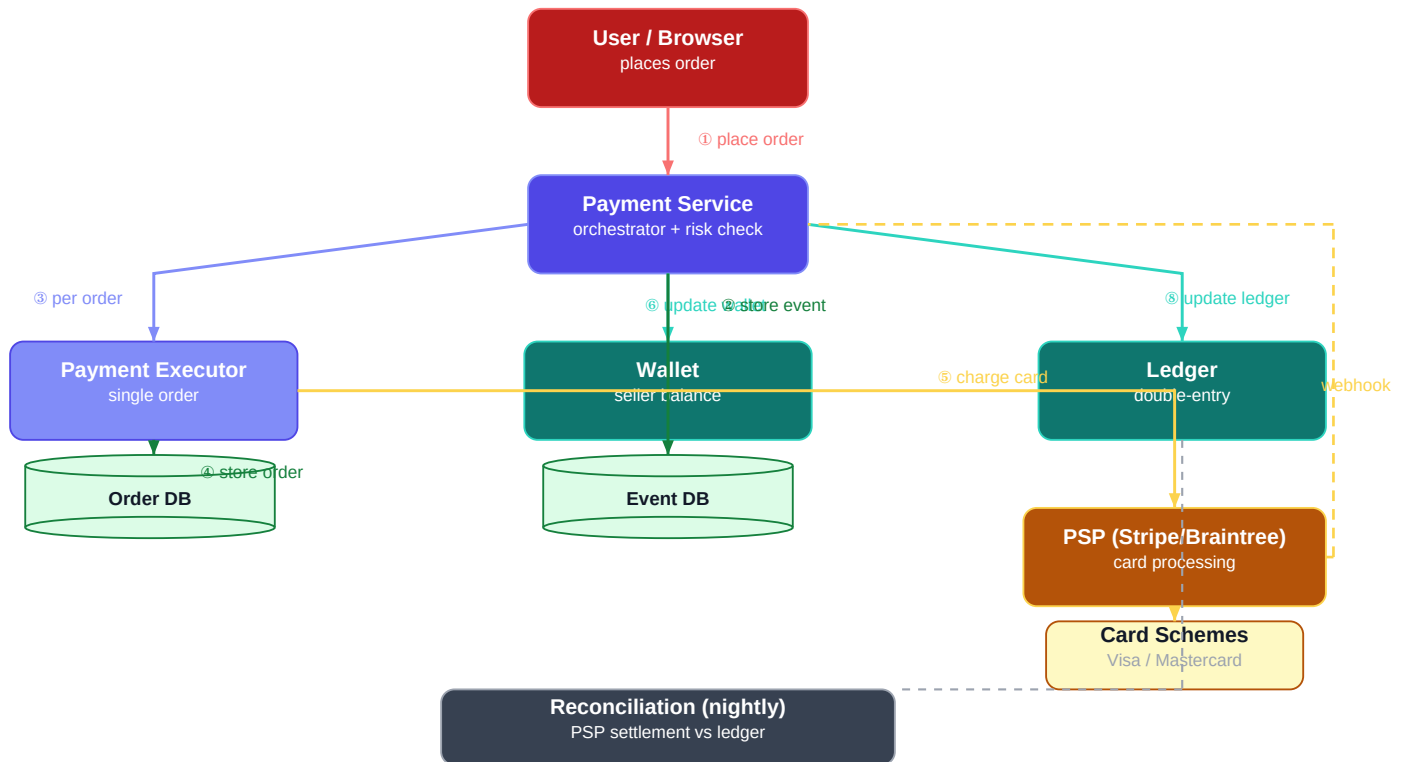


Fig 1 — Overall payment system architecture

Core Components

Component	Role & Responsibilities
Payment Service	Central orchestrator. Accepts payment events, runs AML/CFT risk check via third-party, coordinates Executor, Wallet, and Ledger. Owns the payment event state machine.
Payment Executor	Executes ONE payment order via PSP. A single payment event may spawn multiple executors (multi-seller cart). Stores per-order execution status.
PSP (Stripe, etc.)	Moves money from buyer's credit card to platform bank. Provides hosted payment page so card data never touches our servers. Returns token + webhook on completion.
Card Schemes	Visa, Mastercard, Discover — connected by the PSP, not directly by us. Complex interchange/dispute ecosystem outside our system boundary.
Wallet	Tracks per-merchant account balance. Updated after PSP confirms successful payment.
Ledger	Immutable double-entry accounting log. Every transaction = debit + credit. Sum always = 0.
Reconciliation	Nightly job comparing PSP settlement file vs internal ledger. Flags and fixes mismatches.

3. Pay-In Flow — Step by Step

The pay-in flow starts when a buyer clicks "Place Order". One checkout event can contain multiple payment orders (e.g. products from different sellers). Each order is executed independently by the Payment Executor against the PSP.

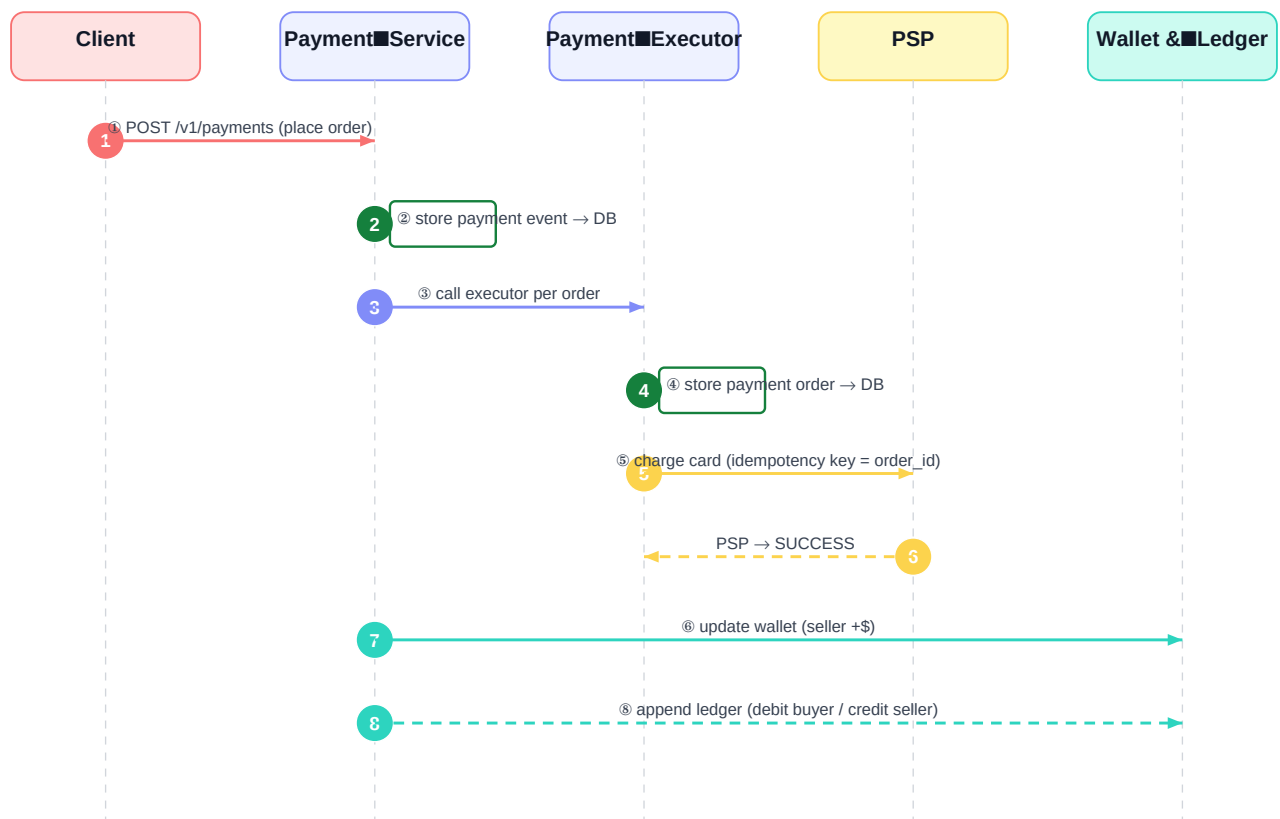


Fig 2 — Pay-in sequence: Client → Payment Service → Executor → PSP → Wallet/Ledger

Step-by-Step Breakdown

Step	Actor	Action & Notes
①	Client → Payment Service	POST /v1/payments with payment_orders array. Idempotency-key header (cart UUID) prevents double submission.
②	Payment Service	Validate, run AML/CFT risk check. Store payment_event in DB (is_payment_done=FALSE). Reject if risk check fails.
③	Payment Service → Executor	Call Payment Executor once for EACH order in the event. Multi-seller cart = multiple executor calls.
④	Payment Executor	Store payment_order in DB (status=EXECUTING). payment_order_id serves as PSP idempotency nonce.
⑤	Executor → PSP	Call PSP to charge buyer's card. Idempotency key prevents double-charge on retry.
⑥	PSP → Executor	PSP returns SUCCESS or FAILED. Executor updates payment_order_status in DB.
⑦	Payment Service → Wallet	On SUCCESS: credit seller balance. Set wallet_updated=TRUE.
⑧	Payment Service → Ledger	Append double-entry record. Set ledger_updated=TRUE. If all orders done: is_payment_done=TRUE.

API Design

Endpoint	Method	Description
----------	--------	-------------

/v1/payments	POST	Execute a payment event (may contain multiple orders). Requires idempotency-key header.
/v1/payments/{id}	GET	Return execution status of a single payment order by payment_order_id.

Request Body Fields

Field	Type & Notes
buyer_info	Object — buyer name, email, address (NOT card data)
checkout_id	String (UUID) — unique ID for this checkout; used as idempotency key at event level
payment_orders[].payment_order_id	String (UUID) — globally unique; used as PSP nonce/idempotency key
payment_orders[].seller_account	String — target seller wallet or bank account
payment_orders[].amount	String (NOT double!) — avoids floating-point rounding errors across systems
payment_orders[].currency	String — ISO 4217 code (e.g. "USD", "EUR", "INR")

Why string for amount? Double/float causes rounding errors across different protocols, languages, and hardware. Store amounts as strings, parse only for display. Example: Japan's GDP ~5×10¹⁴ yen; Bitcoin satoshi = 10⁸. Both extremes need precise string representation.

Data Model

Table	Field	Type & Notes
payment_event	checkout_id	PK — unique per checkout session
	buyer_info	String — stored buyer details
	is_payment_done	Boolean — TRUE when ALL orders in this event succeed
payment_order	payment_order_id	PK — globally unique UUID; also PSP nonce
	checkout_id	FK → payment_event
	amount	String — see note above
	currency	ISO 4217 string
	payment_order_status	ENUM: NOT_STARTED EXECUTING SUCCESS FAILED
	wallet_updated	Boolean — TRUE after wallet credited
	ledger_updated	Boolean — TRUE after ledger appended

4. Pay-Out Flow

During pay-in, money lands in the **platform's bank account** — the platform acts as a custodian. The seller doesn't directly receive money yet. After the delivery condition is met, a pay-out is initiated, transferring the seller's share from the platform bank to the seller's own bank account.

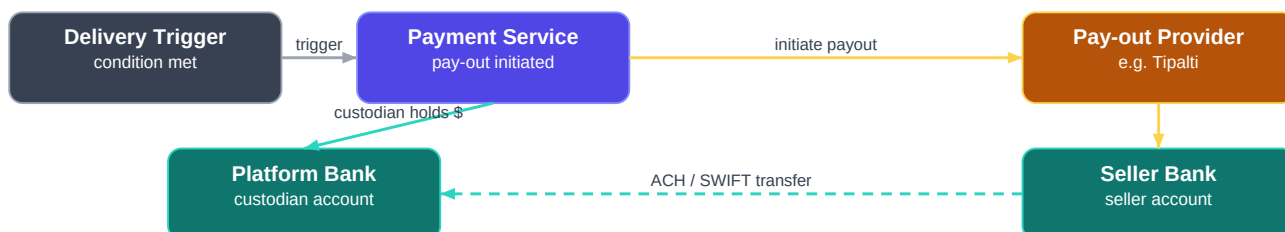


Fig 3 — Pay-out flow: Platform bank → Seller bank via third-party payout provider

Stage	Description
1. Delivery trigger	Product delivered (or time-based condition met). Pay-out is authorised.
2. Wallet check	Payment Service reads seller balance from Wallet to determine payout amount.
3. Payout provider	Third-party (e.g. Tipalti) handles bank transfer, tax forms, FX conversion, local rail selection (ACH, SWIFT, SEPA), and compliance per country.
4. Bank transfer	Money moves: Platform bank → Seller bank. Typically T+1 or T+2 settlement.
5. Ledger update	Double-entry: debit platform custodial account, credit seller account.

Pay-in vs Pay-out: Pay-in uses PSP to charge the buyer's card (money → platform bank). Pay-out uses a separate payout provider (Tipalti) to push money to the seller's bank. Different providers, different rails, different compliance requirements.

5. Double-Entry Ledger System

The ledger is the financial backbone. Every transaction is recorded as **two equal entries** — one debit and one credit. The sum of all entries must always equal zero. If money disappears, the sum breaks — making bugs and fraud immediately detectable.

Double-Entry Ledger — Buyer pays Seller \$1.00

Account	Debit	Credit
Buyer account	\$1.00	
Seller account		\$1.00
TOTAL (sum=0)	\$1.00	Sum of all entries = \$1.00

Money cannot disappear undetected

Fig 4 — Double-entry ledger: every transaction has matching debit and credit entries

Property	Description
Append-only (immutable)	New entries are always INSERTs, never UPDATEs. Corrections require a new reversing entry. Full audit trail at all times.
Sum = 0 invariant	Sum of all debit entries = sum of all credit entries. Violation = bug or fraud.
Revenue analytics	Total platform revenue, seller earnings, refunds, and fees all derivable from ledger entries.
Reconciliation input	Compared nightly against PSP settlement files. Any mismatch triggers alert or auto-fix.

6. Step 3 Deep Dive — PSP Integration & Hosted Payment Page

Most companies use a **PSP-hosted payment page** (iframe or SDK) to avoid handling card data. This keeps PCI DSS compliance within the PSP's scope. Our system only ever sees an opaque token.

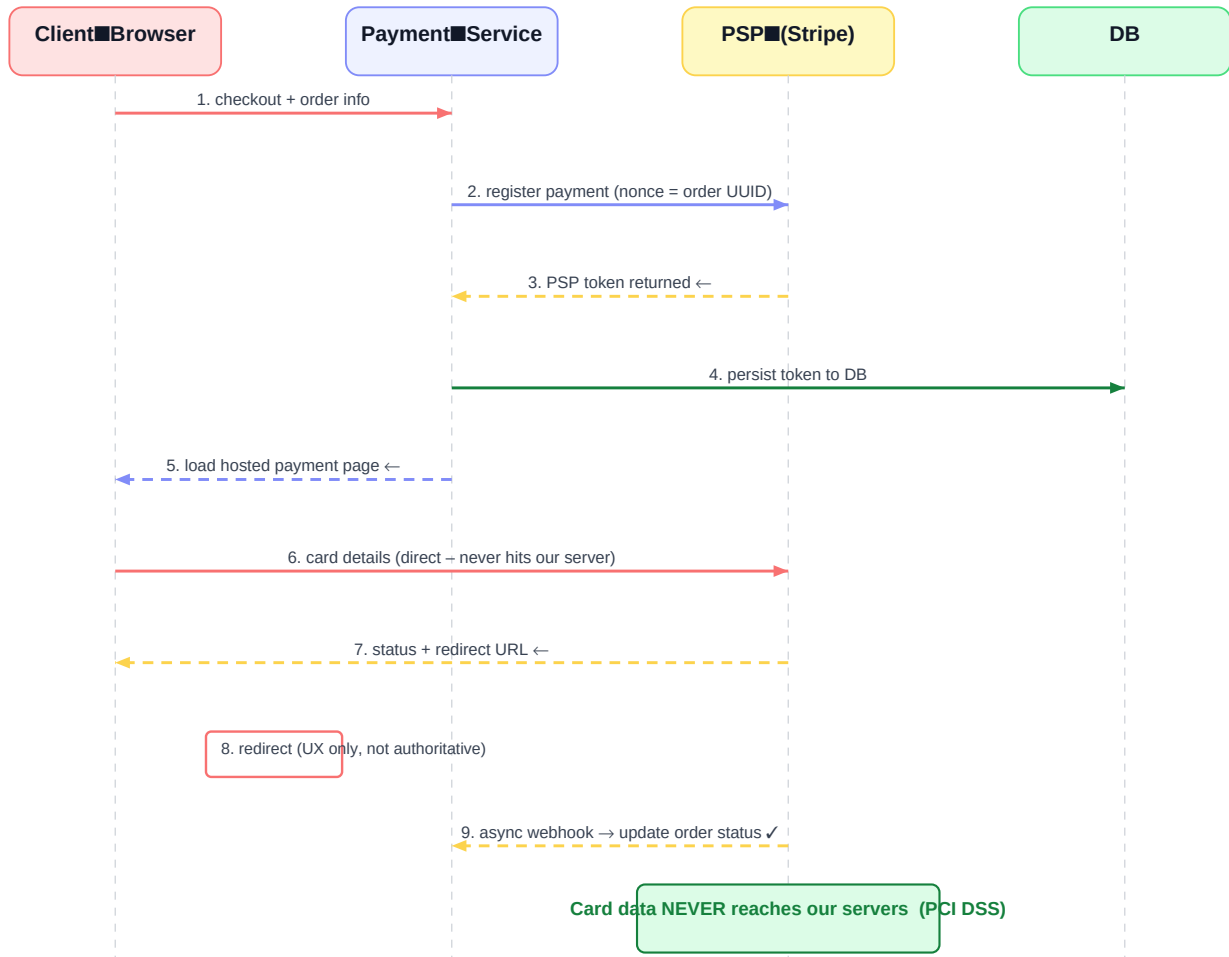


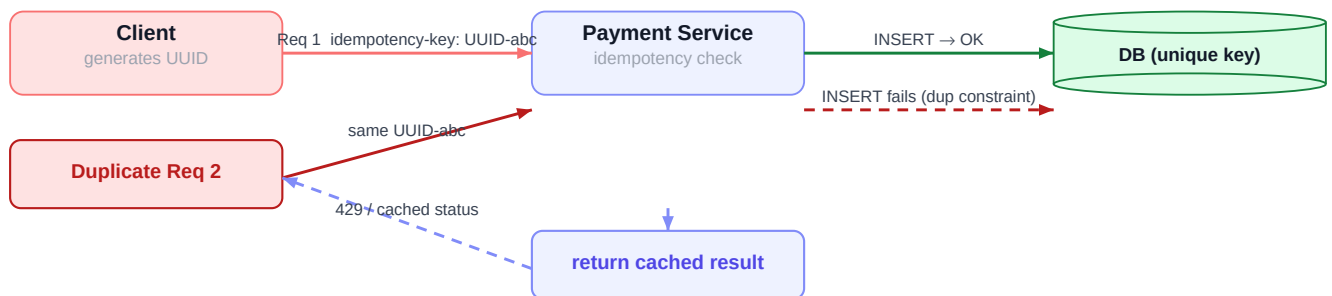
Fig 5 — 9-step PSP hosted payment flow. Card data goes directly to PSP — never to our servers.

Step	Description
1. Client checkout	Browser sends order info to Payment Service.
2. Register with PSP	Payment Service sends: amount, currency, expiry, redirect URL, and nonce (UUID = payment_order_id). Nonce ensures exactly-once registration.
3. Token returned	PSP returns a UUID token identifying this payment registration on PSP side.
4. Persist token	Payment Service saves token to DB BEFORE loading hosted page — protects against client failures.
5. Hosted page	Client loads PSP iframe/SDK (e.g. Stripe.js). Collects card details. Never reaches our servers.
6. Card submitted	Card number, CVV, expiry go directly to PSP. Our system is never in the path of raw card data.
7. PSP processes	PSP charges card via card scheme, returns status + redirect URL.
8. Redirect (UX only)	Browser redirects to merchant page. NOT authoritative — can be spoofed.
9. Webhook (authoritative)	PSP calls our webhook URL async with final status. Update DB here. THIS is the source of truth.

Critical: Always use the WEBHOOK (step 9) as the authoritative payment result. The redirect URL (step 8) is UX only and can be manipulated. Validate PSP webhook signatures (HMAC-SHA256) on every incoming event.

7. Exactly-Once Delivery & Idempotency

Formula: Exactly-once = At-least-once (retry) + At-most-once (idempotency). Both halves are required — retry alone causes duplicates; idempotency alone causes missed payments.



Exactly-once = At-least-once (retry) + At-most-once (idempotency)

Fig 6 — Idempotency key flow: duplicate requests return cached result without re-executing

Retry Strategy	Description & When to Use
Immediate retry	Retry instantly. Only for momentary blips — rarely appropriate in payments.
Fixed interval	Wait constant time (e.g. 1s). Simple but can overload a recovering service.
Incremental backoff	Wait grows linearly. Gentler for short recovery windows.
Exponential backoff	Double wait each attempt: 1s → 2s → 4s → 8s. Standard for payments. Add jitter to avoid thundering herd.
Cancel / give up	After N retries, stop. Non-retryable errors (fraud, invalid card) go here immediately.

Scenario	Solution
User double-clicks Pay	Both requests carry the same idempotency key (cart UUID). Second hits DB unique constraint → return 429 with existing status.
PSP response lost in network	User retries. Same payment_order_id sent to PSP as nonce. PSP returns original charge result — no double charge.
Internal service retry	Same idempotency key on retry. DB constraint or PSP-side idempotency prevents duplicate processing.

8. Handling Failed Payments — Retry & DLQ

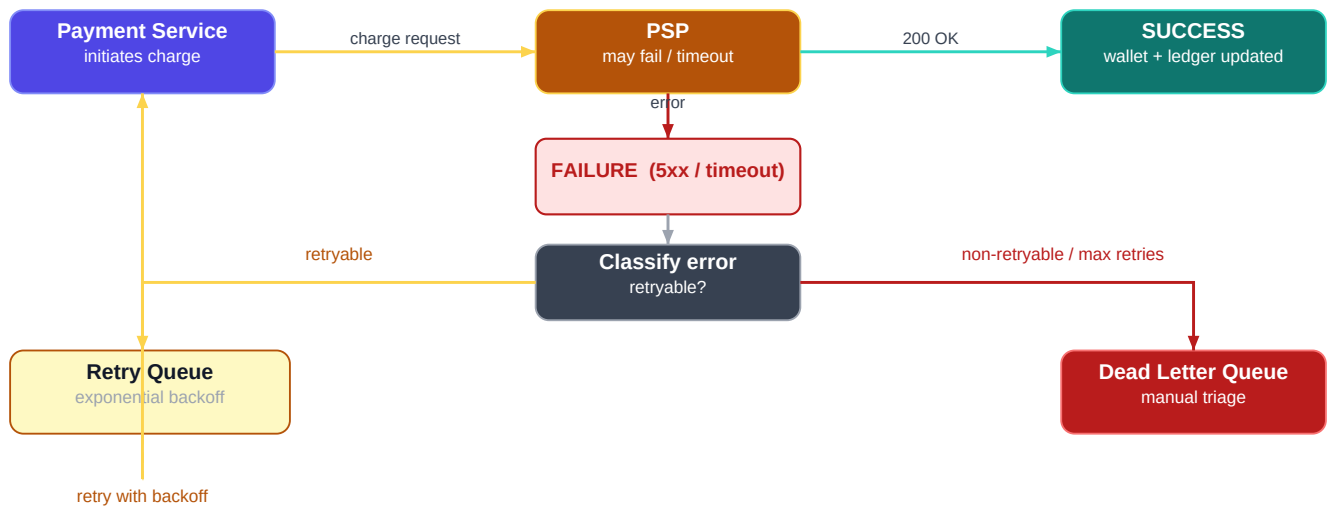


Fig 7 — Retry queue and Dead Letter Queue (DLQ) failure handling flow

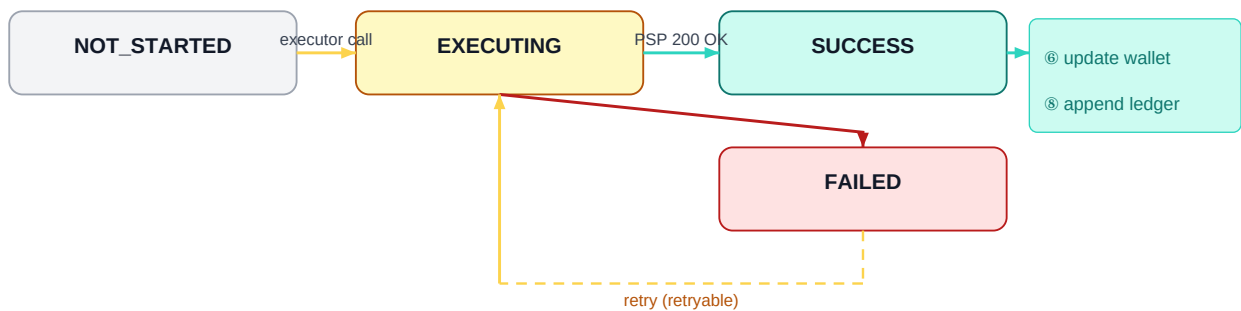


Fig 8 — payment_order_status state machine

Queue / Store	Purpose & Behaviour
Retry Queue	Receives transient failures (5xx, timeouts). Re-processes with exponential backoff. Retry counter tracked. Implemented with Kafka or SQS in practice.
Dead Letter Queue	Messages exceeding max retry count arrive here. Engineers investigate manually. Also receives non-retryable failures for audit trail.
Error store (non-retryable)	Invalid card, fraud, insufficient funds — stored directly in DB. No retry. Customer support alerted.

Status	Transition	Notes
NOT_STARTED	→ EXECUTING	Order created, sent to executor
EXECUTING	→ SUCCESS or FAILED	PSP call in progress
SUCCESS	(terminal)	Wallet + ledger updated; is_payment_done=TRUE when all orders complete
FAILED (retryable)	→ retry queue → EXECUTING	Transient error, exponential backoff applied
FAILED (permanent)	(terminal → DLQ)	Non-retryable; manual investigation required

9. Reconciliation — Last Line of Defense

Because services communicate asynchronously and external systems can fail silently, nightly reconciliation is mandatory. Every night the PSP sends a **settlement file** containing all transactions and the closing balance. The reconciliation service parses this and compares against the internal ledger.

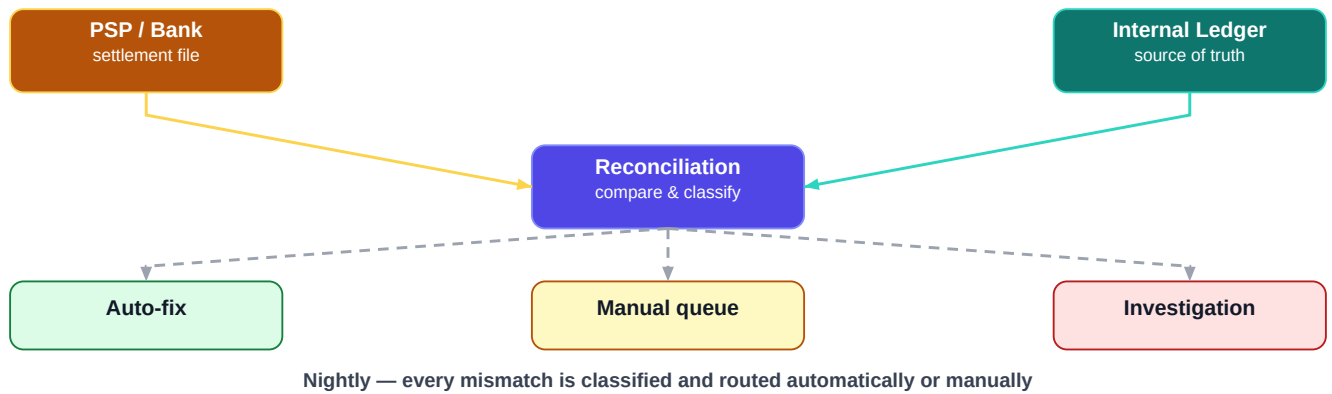


Fig 9 — Reconciliation pipeline: settlement file → compare → classify → fix or investigate

Mismatch Type	Resolution
Classifiable + automatable	Known cause, known fix, cost-effective to code. Auto-adjustment runs on next cycle.
Classifiable + manual	Known cause but automation cost too high (rare edge case). Queued for finance team.
Unclassifiable	Unknown root cause. Special investigation queue. Finance + engineering investigate together.

10. Communication Among Services

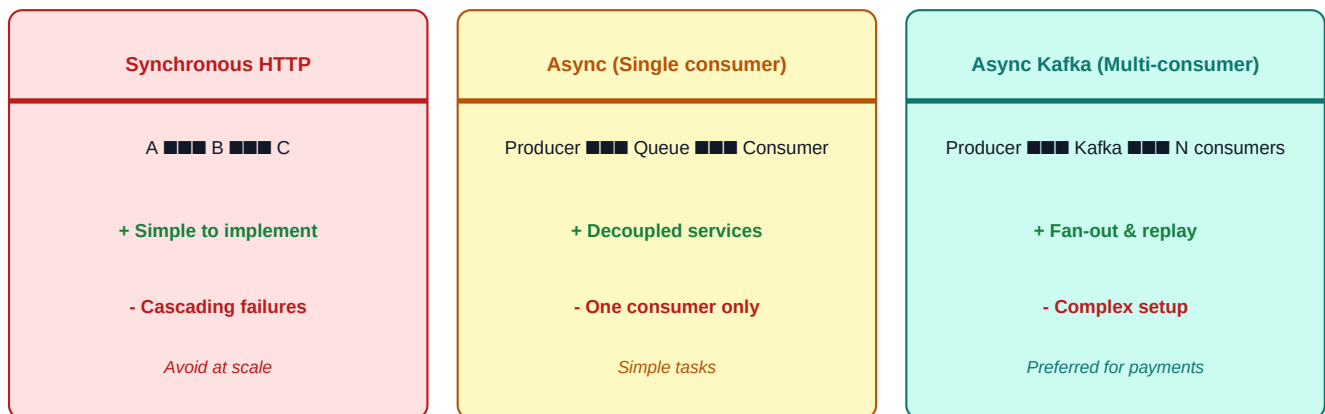


Fig 10 — Three communication patterns compared: sync HTTP, async queue, async Kafka

Pattern	Pros / Cons & When to Use
Synchronous HTTP	Simple to implement and debug. Suitable for small scale. BUT: tight coupling, cascading failures, poor fault isolation — one slow service stalls the whole chain.
Async — single consumer (queue)	Each message processed by exactly one consumer. Good for task distribution. Consumed messages removed from queue. Limited fan-out.
Async — multi-consumer (Kafka)	Same payment event consumed by many services in parallel (payment, analytics, billing, notifications). Messages persist — consumers can replay. Preferred for large-scale systems.

11. Handling Payment Processing Delays

Some payments take hours or days — 3D Secure authentication, high-risk manual review, or slow bank transfers. The system stores them as EXECUTING and relies on webhooks to get the final result asynchronously.

Scenario	Cause	Handling
PSP returns "pending"	Bank transfer or risk review	Show pending to user. PSP tracks and fires webhook on resolution.
3D Secure required	Card issuer extra auth	User redirected to issuer page. PSP waits, then resumes and webhooks us.
High-risk manual review	PSP human review (24-72h)	Store as EXECUTING. Monitor job alerts on stuck orders > threshold.
PSP has no webhook	Some PSPs only support polling	Poll GET /payment/{id} on schedule for EXECUTING orders past threshold.

12. Consistency Strategies

Problem	Solution
Internal state diverges (ledger vs wallet)	Exactly-once processing via idempotency + append-only transitions. Scheduled monitor alerts on stuck orders.
External PSP mismatch	Idempotency keys on all PSP retries + nightly reconciliation.

DB replication lag	Option A: Read/write from primary only. Option B: Consensus DB (CockroachDB, YugabyteDB).
DB technology	Prefer ACID relational (PostgreSQL, MySQL) over NoSQL. Stability, tooling, and DBA market matter more than throughput at 10 TPS.

13. Payment Security

Control	Implementation
HTTPS / TLS 1.3	All traffic encrypted in transit. Mutual TLS between internal microservices.
Encryption at rest	AES-256 for sensitive DB fields. Keys via HSM or cloud KMS (AWS KMS, GCP KMS).
PCI DSS compliance	Delegated to PSP-hosted page. We never see raw card numbers, CVVs, or expiries.
Tokenization	PSP replaces card number with opaque token. Stored for future charges. Cannot be reverse-engineered.
AML / CFT risk check	Every payment screened by third-party risk service before processing. Failed checks rejected before PSP.
Rate limiting	Payment endpoints throttled. Return 429 + Retry-After. Per-user and per-IP limits. WAF in front.
Idempotency keys	UUID per payment prevents duplicate charges. DB unique constraint enforces exactly-once.
Webhook signatures	PSP signs payload (HMAC-SHA256). Reject any event failing signature verification.
Fraud detection (ML)	Velocity checks, geo mismatch, unusual amounts. Real-time scoring before PSP call.

14. Interview Cheat Sheet — Q&A;

How do you prevent double charges?

Idempotency key (UUID) on every request header. DB unique constraint on `payment_order_id`. PSP-side nonce prevents duplicate PSP calls. Reconciliation is the safety net.

What database would you use and why?

Relational (PostgreSQL / MySQL) with ACID transactions. At 10 TPS, correctness, stability, rich tooling, and DBA availability matter far more than throughput. NoSQL's eventual consistency is incompatible with financial data integrity.

How do you handle payment failures?

Retry queue with exponential backoff for transient errors. Dead letter queue for messages exceeding max retries. Non-retryable errors (invalid card, fraud) stored directly — no retry. Scheduled monitor alerts on orders stuck in EXECUTING past a threshold.

How do you handle card data / PCI DSS?

We never store card numbers. PSP-hosted page (iframe/SDK) captures card data directly. Our system only receives an opaque PSP token. PCI DSS compliance fully delegated to PSP.

How do you ensure consistency across services?

Internal: exactly-once via idempotency + append-only state transitions. External: idempotency keys on PSP retries + nightly reconciliation. DB: read/write primary or consensus-based DB (CockroachDB, YugabyteDB).

Explain the ledger system.

Double-entry bookkeeping: every transaction = debit + credit of equal amount. Sum = 0 always. Append-only (immutable). Any deviation = bug or fraud. Used for audits, revenue analytics, reconciliation input.

Sync or async communication?

Async with Kafka for scale. Same payment event triggers payment, analytics, billing, notifications in parallel — no tight coupling, better fault isolation, easy replay. Sync causes cascading failures at scale.

How do reconciliation and webhooks work together?

Webhook is the real-time authoritative signal from PSP. Reconciliation is the nightly safety net that catches anything the webhook missed. Both are required — webhooks alone can be lost; reconciliation alone is too slow.

Why store amount as string?

Double/float causes rounding errors across systems. Japan's GDP $\sim 5 \times 10^{14}$ yen and Bitcoin satoshi = 10^8 both need precise representation. Store as string, parse only for display or arithmetic.

What happens if PSP is slow (processing delays)?

Store order as EXECUTING. PSP tracks the payment and fires webhook when it resolves. Scheduled monitor job alerts if any order stays EXECUTING beyond a configured time threshold.

